A11｜怎样保存和找回一份确定的代码?

A10 观察一次运行；A11 为运行结果补上内容身份：**改的是哪份代码，准备保存哪份，怎样找回已确认的版本。**

怎样把修改变成明确记录?

当前问题	关系或短用法	记到什么程度
编辑器保存了，为什么还没有历史?	保存改变 working tree（工作区）；`git add` 选取当时的内容放入 stage/index（暂存区）； `git commit` 把暂存内容记入本地历史。	**记住三个位置与交接，逐渐用熟 add、commit。** 它们不自动替彼此完成下一步。
哪些文件变了，具体变了什么?	`git status --short` 看状态；普通 `git diff` 比较工作区与暂存区；`git diff --staged` 比较暂存区与当前提交。	**认熟两种 diff 看的是不同一段。** `--` 后接路径，可将比较限制到明确文件。
add 后又编辑，提交保存谁?	普通 commit 保存暂存内容，不自动收进后来编辑的文字。程序和测试通常读工作区，所以刚测试通过不一定证明暂存内容正确。	沿三份内容判断提交结果，不只记“add 后接 commit”。
日志里的编号有什么用?	`git log --oneline` 查提交历史；提交 ID 定位具体提交，`HEAD` 在本章场景指当前检出的提交。	认熟日志入口；编号用来记录、核对，不背。commit 保存快照，diff 比较两份内容。

表内省略了文件、分支等参数的写法用于识别命令，不一定能原样执行；实际操作按对应正文取得完整指令，并先核对目标。

例如：当前提交是 `Hello, learner`；改为 `Hello, team` 后 add；又改为 `Hello, everyone` 并保存。运行读到的是 `everyone`，普通 commit 准备保存的却是 `team`。

回看：「先选中内容，再把它记入历史」「add 以后继续编辑，下一次提交会得到哪一份？」。

不提交、放弃修改、纠正提交，各改哪里?

当前问题	关系或短用法	记到什么程度
暂时不提交这个文件，但保留编辑	`git restore --staged` 改暂存区；默认从当前提交取内容，工作区不变。	理解"取消待提交选择"不是删除编辑内容；完整命令带核对过的文件路径查用。
确认未提交编辑不要了	普通 `git restore` 默认从暂存区恢复工作区，覆盖对应文件的修改，不建立备份提交。	**先看 status 和 diff，说清来源、目标及要放弃的内容。** 不把单文件恢复扩大成整目录操作。
改错内容已经提交了	`git revert` 反向应用指定提交的改变并形成新提交；旧提交保留，过程中可能需要处理冲突。	区分未提交恢复与已提交纠正；目标提交与影响先核对，完整步骤查用。
想另开修改方向，再合回来	branch 是指向提交的名字；`git switch` 切换， `git merge` 整合。冲突要按最终需求决定内容。	理解历史怎样分开、汇合；切换不等于自动保存或清空未提交修改。

表中的恢复写法是动作名称，不是让你直接运行的完整指令。回看「只是不想提交，和想丢掉修改，是两个动作」「如果错误已经提交，怎样保留纠正的经过?」「两个人从同一份内容继续改，历史怎样分开?」。

本地保存以后，怎样分享?

当前问题	关系或短用法	记到什么程度
本地提交后，远端为什么没变?	`git fetch` 取回远端信息，不自动合入当前分支；`git pull` 取回后整合；`git push` 请求远端接收提交、更新相应引用。	**commit 不等于上传，未提交文字不会随 push 上传。** 分支和参数按项目查。
哪些内容不该跟着分享?	`.gitignore` 主要影响尚未跟踪的匹配文件，不让已跟踪内容或历史秘密消失。	分享前看暂存差异及受跟踪清单；已泄露密钥须撤销或轮换，不靠改文件名解决。

回看: 「本地提交以后，另一台机器为什么还没有变化?」「准备分享时，哪些文件不该进入这份历史?」。新实验与检查配方见「在独立目录中完成这次保存与恢复」，不需要默写整段。

主关系是**比较 → 选择内容 → 确认提交；恢复前说清从哪里取、改哪里**。A12 再验证这份确定的内容，能否在新环境重新工作。